

autolisp-test Harness

Codex

June 24, 2026

The harness is written in pure AutoLISP and depends only on standard AutoLISP / Visual LISP features available on every supported target (AutoCAD, BricsCAD, clautolisp).

Files

rt.lsp registration core. Defines `deftest`, `deftest-eq`, `deftest-pred`, `deftest-error`, `deftest-skip`. Maintains the global `*autolisp-tests*` registry. Implements `autolisp-test-classify` which evaluates a test's form, captures errors via `vl-catch-all-apply`, and returns a classification alist.

profiles.lsp canonical profile (STRICT / AUTOCAD / BRICSCAD) and tag sets (platform: WINDOWS / LINUX / MACOS; runtime: COM VLA VLAX VLAX-CURVE VLR ARX BRX OBJECTDBX DCL GRAPHICS USER-INPUT EXPRESS-TOOLS DOSLIB). Computes per-subset verdicts (CONFORMS / DEVIATES / NOT-APPLICABLE) and the global verdict matrix.

platform-detect.lsp builds the implementation descriptor: IMPL, IMPL-NAME, VERSION, PLATFORMS, RUNTIMES, PROFILE-TARGET. Detects clautolisp via the `*clautolisp-version*` marker; AutoCAD and BricsCAD via (`getvar "PRODUCT"`) and (`getvar "ACADVER"`). Detects platform via OS, OSTYPE and (`getvar "PLATFORM"`). Detects runtime extensions by probing well-known function names (`vlax-create-object`, `vla-get-application`, `vlr-object-reactor`, `arxload`, `brxload`, `entget`, `load_dialog`, `grdraw`, `getpoint`, `acet-file-copy`, `dos_alert`).

report.lsp evaluates every applicable test, applies expectation overlays, computes the verdict matrix, writes the canonical s-expression report (one record per line) and prints a human-readable recap.

expectations.lsp expected-failure overlay loader. Looks up `harness/expectations/<impl>/<version>` and reclassifies `FAIL -> XFAIL`, `PASS -> XPASS`, etc.

test-loader.lsp the explicit manifest of every test file under `tests/`.
Adding a new test file is a two-step operation: drop the file under `tests/`, then add a line to `*autolisp-test-files*`.

run.lsp top-level entry point. Loads the harness modules, the test corpus, detects the implementation, runs everything, writes the report, prints the recap, and returns a result alist.

inventory.sexp machine-readable inventory of every spec entry (976 records).
The pre-classification (default profile, default tags, default authority) is documented in the file header. Refinements as AutoCAD and BricsCAD probes accumulate.

Running

The `autolisp-test/Makefile` provides four targets covering both launch paths and both supported Common Lisp implementations, plus their debug variants:

Target	Path	Prerequisite
<code>make test-clautolisp-sbcl</code>	standalone executable	<code>make -C clautolisp build</code>
<code>make test-clautolisp-ccl</code>	standalone executable	<code>make -C clautolisp build</code>
<code>make test-asdf-sbcl</code>	SBCL + ASDF driver	none
<code>make test-asdf-ccl</code>	CCL + ASDF driver	none
<code>make test-clautolisp-sbcl-debug</code>	standalone executable, debug	(interactive only)
<code>make test-clautolisp-ccl-debug</code>	standalone executable, debug	(interactive only)
<code>make test-asdf-sbcl-debug</code>	SBCL + ASDF driver, debug	(interactive only)
<code>make test-asdf-ccl-debug</code>	CCL + ASDF driver, debug	(interactive only)

The umbrella `make test` runs `test-clautolisp-sbcl`. That choice makes the canonical run use the same load path that AutoCAD and BricsCAD take when they (`load "...run.lsp"`) themselves, so the verdict matrix is comparable across implementations.

A normal run never drops into the host Lisp debugger, regardless of how the tested implementation behaves: errors raised inside a tested function are caught by `vl-catch-all-apply` and reported as test failures (or as expected error matches when the test is a `deftest-error`); errors that escape the

harness logic itself are caught by the outer per-test guard and reported as **internal-harness-error** test failures; any error that escapes both guards is intercepted by the CL-side handler and printed on stderr without entering the debugger.

Failure details include an AutoLISP backtrace whenever the host exposes one. clautolisp's runtime tracks ***autolisp-call-stack*** and captures the snapshot at error time; the harness reads it back via **vl-catch-all-error-stack** (a clautolisp extension) and embeds it in the failure detail and in the s-expression report under the **stack** key. AutoCAD and BricsCAD do not expose the stack of a caught error, so the harness simply omits it on those targets. The (**vl-bt**) builtin prints the live stack at the point of call and works on any host that implements it.

The **-debug** variants set ***autolisp-test-debug-p*** to T (and pass **:debug t** to the CL driver), which is a **verbosity flag** rather than an error-propagation switch. In debug mode the harness prints **[autolisp-test]** **>>> NAME form: FORM** before every test and echoes each FAIL detail (including the captured AutoLISP backtrace) on stdout immediately after the test. Errors are still caught, the run still completes, and the report directory is still produced.

The report is written under **autolisp-test/results/<impl>/<version>/<host>/<timestamp>/r**

Path 1: standalone clautolisp executable

After **make -C clautolisp build-clautolisp-sbcl**, the Makefile target runs:

```
clautolisp/tools/clautolisp/bin/clautolisp-sbcl -x \
  '(setq *clautolisp-version* "0.x") \
    (setq *autolisp-test-root* "<repo>/autolisp-test/") \
    (load (strcat *autolisp-test-root* "harness/run.lsp")) \
    (autolisp-test-run-all)'
```

The ***clautolisp-version*** **setq** is required: the standalone CLI does not bind it, so **platform-detect** would otherwise fall back to implementation **UNKNOWN**. The ASDF driver path below sets it automatically.

Path 2: SBCL/CCL + ASDF driver

The driver installs the core builtins, exports the ***clautolisp-version*** marker so **platform-detect** identifies the implementation, and invokes the AutoLISP harness inside a fresh runtime session:

```
(asdf:load-system "autolisp-test/clautolisp-driver")
(autolisp-test.clautolisp-driver:run-autolisp-test-suite)
```

Inside any conforming AutoLISP environment (AutoCAD, BricsCAD)

Load `run.lisp` directly from the editor:

```
(setq *autolisp-test-root* "<absolute-path-to>/autolisp-test/")
(load (strcat *autolisp-test-root* "harness/run.lisp"))
(autolisp-test-run-all)
```

Platform-detect identifies the implementation via `(getvar "PRODUCT")` and does not need any extra marker.

Reports

Each report is a sequence of single-line s-expressions:

```
((RUN-START) (IMPL . CLAUTOLISP) (IMPL-NAME . "clautolisp") ... )
((TEST-RESULT) (NAME . "if-true-then-branch") (STATUS . PASS) ... )
((TEST-RESULT) (NAME . "if-nil-else-branch") (STATUS . PASS) ... )
...
((VERDICT) (SUBSET . "strict") (VERDICT . CONFORMS) (PASS . 980) (FAIL . 0) ... )
((VERDICT) (SUBSET . "autocad") (VERDICT . NOT-APPLICABLE) ... )
((VERDICT) (SUBSET . "bricscad") (VERDICT . NOT-APPLICABLE) ... )
((RUN-END) (IMPL . CLAUTOLISP) ... )
```

Reports are diffable across implementations and across runs. `tools/diff-reports.lisp` aligns N reports and surfaces every divergence with the offending test name and concerned profile/tag combination.

Authority

Every test entry carries an `:authority` meta value:

DOCUMENTED backed by vendor documentation. Default. Kept distinct from **TESTED-*** because vendor documentation has been observed to diverge from actual product behaviour.

TESTED-AUTOCAD confirmed by an executable probe against a named AutoCAD version.

TESTED-BRICSCAD confirmed by an executable probe against a named BricsCAD version.

TESTED-BOTH both vendor probes agree.

INFERRED derived from sources or strong consistency arguments when no direct documentation or probe is available.